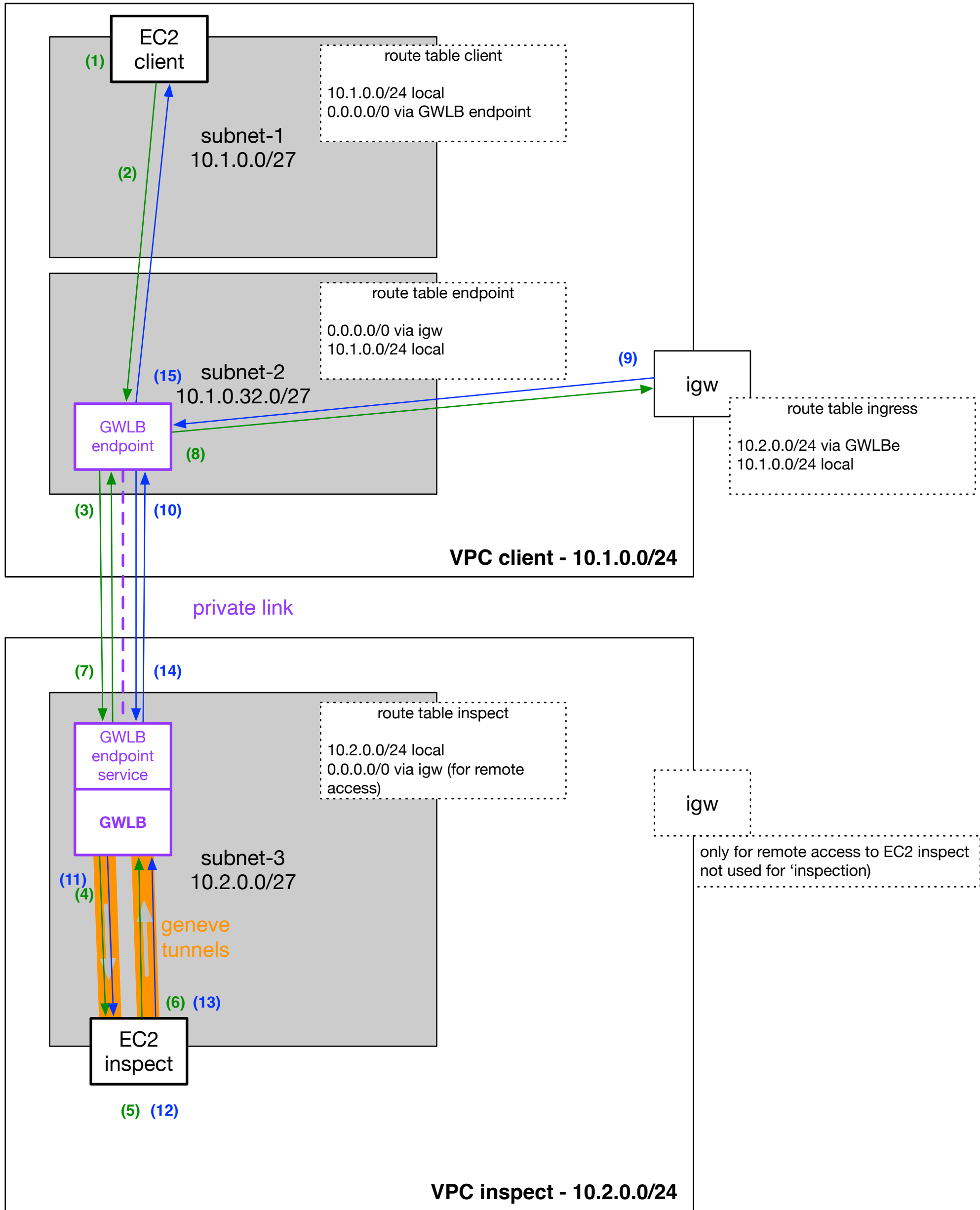




Outbound traffic

- (1) the packet from EC2 client is routed toward VPC router in subnet-1 / route table client
- (2) the packet is routed towards GWLB endpoint per default route in subnet-1 / route table client
- (3) packet is routed via private link between GWLB endpoint in VPC client to GWLB in VPC inspect
- (4) GWLB forwards the packet via GENEVE tunnel to “EC2 inspect”
- (5) per routing policy on “EC2 inspect”, the packet is routed back with outbound GENEVE tunnel
- (6) outbound GENEVE tunnel brings it back to GWLB
- (7) GWLB returns the packet back via private link from VPC inspect to VPC client
- (8) GWLB endpoint routes the packet toward igw per default route in route table endpoint

(note: as EC2 client as public IP assigned, the NAT to public IP happens on igw)

Return traffic

- (9) igw has *specific* route table ‘ingress’ (!) and the return packet if forwarded to GWLB endpoint
- (10) return packet is routed via private link between GWLB endpoint in VPC client to GWLB in VPC inspect
- (11) GWLB forwards the return packet via GENEVE tunnel to “EC2 inspect”
- (12) per routing policy on “EC2 inspect”, the return packet is routed back with outbound GENEVE tunnel
- (13) outbound GENEVE tunnel brings return packet back to GWLB
- (14) GWLB returns the return packet back via private link from VPC inspect to VPC client
- (15) finally, the return packet is routed towards EC2 client per route-table endpoint

note: the general idea is that we need to introduce two new route tables into ‘to-be-inspected’ VPC as in AWS, subnets are tight to route-tables, we need to create new subnet for ‘both direction’: the route table client is for direction ‘out’ and route table ingress for direction ‘in’ within these route table with specify, that the “rest of the world” (out) or the “to-be-inspected subnet” (in) are actually located behind gateway load balancer, and would leave the rest on the GWLB